

UNIVERSIDAD SANTO TOMÁS SECCIONAL TUNJA
Facultad de Ingeniería de Sistemas

DESARROLLO SEGURO DE SOFTWARE

Laboratorio No. 7 · Usuarios, Hashing y Salting · API_TEATRO

ESPACIO ACADÉMICO:
Seguridad en el Desarrollo de Software

PRESENTADO A:
Sergio Arley Puerto Moreno

AUTORES:
Juan Sebastián Bonilla León
Juan Camilo Calderón Delgado
Silvana Sofía Siza Soriano
Cristian Emanuel Hernández Araque
Cristian Rodrigo Amaya Torres

2026

1. El problema

Hasta el Laboratorio 6 la API autenticaba al cliente que hacía cada petición con la cabecera `X-API-Key`. Con las tres claves registradas, Postman Laboratorio, Taquilla del Teatro y Aplicación Móvil, resolvimos qué aplicación consumía `/api`, pero seguíamos sin saber qué persona estaba detrás.

Si alguien vendía una boleta desde la taquilla o programaba una función, el servidor sabía que la petición venía de Taquilla del Teatro, pero no podía distinguir qué trabajador estaba en el mostrador ni qué rol tenía.

En este laboratorio agregamos el segundo nivel de identidad: cuentas de usuario con correo y contraseña, guardando las contraseñas con bcrypt. El proyecto de referencia usa los roles administrador, médico y paciente; en el Teatro Maldonado los adaptamos a los tres actores reales del dominio: administrador, taquilla y asistente.

2. Instalación de bcrypt

Para manejar contraseñas instalamos bcrypt. A diferencia de `crypto`, que viene con Node, esta sí es una dependencia externa, así que justo después de instalarla corrimos `npm audit` y `npm list bcrypt` para ver qué pasó con la superficie de dependencias.

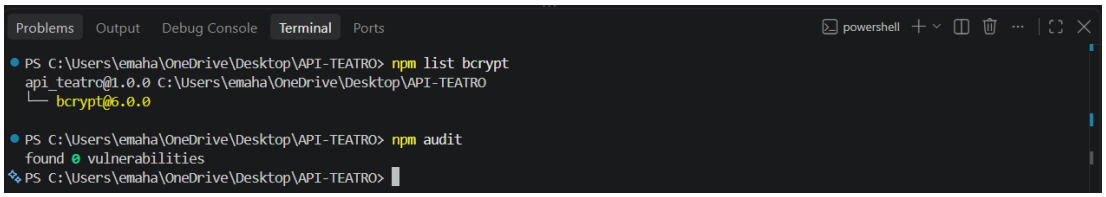


Figura 1. Instalación de bcrypt 6.0.0 y verificación con npm audit: cero vulnerabilidades.

El total pasó de 143 a 146 paquetes. bcrypt trajo una sola dependencia propia, node-addon-api, y ninguna vulnerabilidad.

3. Por qué bcrypt con sal y no SHA-256

En el laboratorio anterior guardamos las API Keys con SHA-256 sin sal, y ahí estaba bien. Esas claves son 32 bytes aleatorios generados criptográficamente, o sea 64 caracteres hexadecimales, y tienen tanta entropía que no se pueden adivinar probando combinaciones ni con tablas precalculadas.

Las contraseñas que escriben las personas son lo contrario: tienen poca entropía, siguen patrones previsibles y la gente suele repetirlas. Si usáramos SHA-256 para contraseñas tendríamos dos problemas.

- SHA-256 está diseñado para ser rápido, así que si el hash se filtra un atacante puede probar millones de contraseñas por segundo.
- Al ser determinista y sin sal, dos usuarios con la misma contraseña producirían exactamente el mismo hash, y eso se vería a simple vista.

Con bcrypt y un factor de trabajo de 12, el algoritmo genera solo una sal aleatoria de 22 caracteres para cada contraseña y la incrusta en la misma cadena del resultado. No hace falta una columna aparte para la sal, cada verificación es deliberadamente lenta, y dos contraseñas idénticas terminan con hashes completamente distintos.

4. Las capas nuevas

Repartimos la autenticación en seis archivos, siguiendo la misma separación de los laboratorios anteriores.

- `src/data/usuarios.js` simula la tabla que más adelante irá en PostgreSQL. Arranca como un array vacío. No dejamos ningún usuario ni contraseña escritos en el código: el primero se crea desde el endpoint de registro.
- `src/utils/password.util.js` encapsula bcrypt con el cost en 12 y expone `generarPasswordHash` y `verificarPassword`. Las dos son asíncronas porque bcrypt es lento a propósito: con las versiones síncronas, cada registro o login bloquearía el hilo principal de Node.
- `src/services/usuarios.service.js` define los roles del teatro, busca usuarios por correo en minúsculas o por identificador, crea usuarios guardando solo el hash, y comprueba credenciales.
- `src/middlewares/auth.validator.js` valida la entrada. En el registro exige nombre de 3 a 100 caracteres, correo válido, contraseña entre 10 y 72 caracteres y un rol de la lista blanca.
- `src/controllers/auth.controller.js` lee el cuerpo solo con `matchedData` y arma la respuesta campo por campo. El `passwordHash` no sale nunca.
- `src/routes/auth.routes.js` define las dos rutas con sus esquemas de OpenAPI, y `app.js` monta el router en `/api/auth`.

Un detalle del validador

Usamos `normalizeEmail()`, que es lo que pide el laboratorio, pero conviene saber que no solo pasa el correo a minúsculas. En Gmail también le quita los puntos al nombre, así que `juan.perez@gmail.com` se guarda como `juanperez@gmail.com`. Para Gmail son la misma

cuenta, así que no rompe nada, pero el correo guardado no siempre es idéntico al que escribió la persona.

Los dos niveles de control

Como `app.use("/api", validarApiKey)` está registrado antes de todos los routers, las dos rutas de `/api/auth` también exigen API Key. Quedan dos niveles: la cabecera dice qué aplicación llama, y el correo con la contraseña dicen qué persona la utiliza.

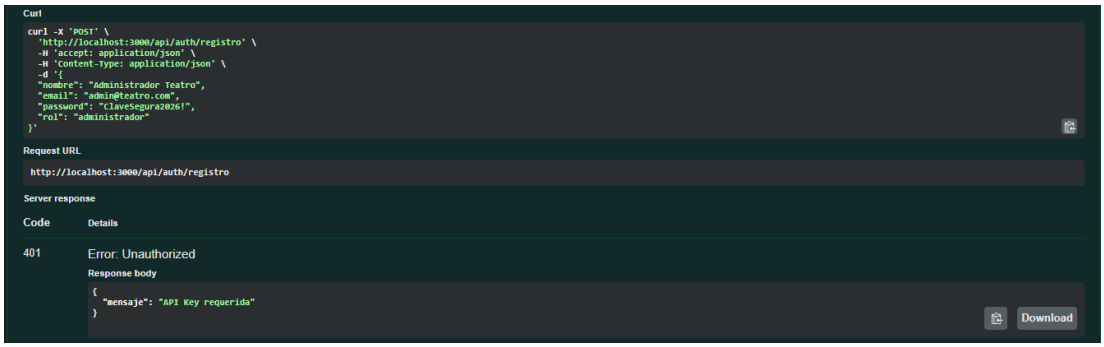


Figura 2. Un registro sin la cabecera X-API-Key responde 401 "API Key requerida": la petición ni siquiera llega al controlador de autenticación.

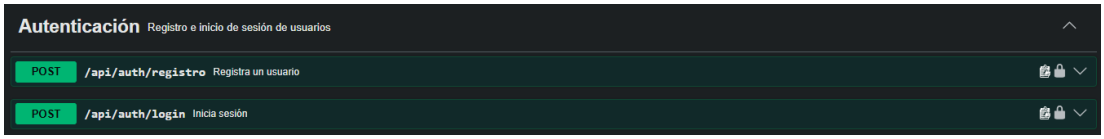


Figura 3. El grupo Autenticación en Swagger, con los endpoints de registro e inicio de sesión.

5. Registro e inicio de sesión

Con el servidor en marcha y tras pulsar Authorize en Swagger con la clave de Postman Laboratorio, probamos todos los estados del flujo.

Registro válido, 201

Registramos el primer usuario: Administrador Teatro, correo `admin@teatro.com`, contraseña `ClaveSegura2026!` y rol administrador. El servidor respondió 201 devolviendo únicamente id, nombre, email, rol y `activo: true`.

Lo importante de esa respuesta es lo que no aparece: ni `password` ni `passwordHash`. Un hash no es la contraseña, pero sigue siendo información sensible que se puede atacar sin prisa si se filtra.

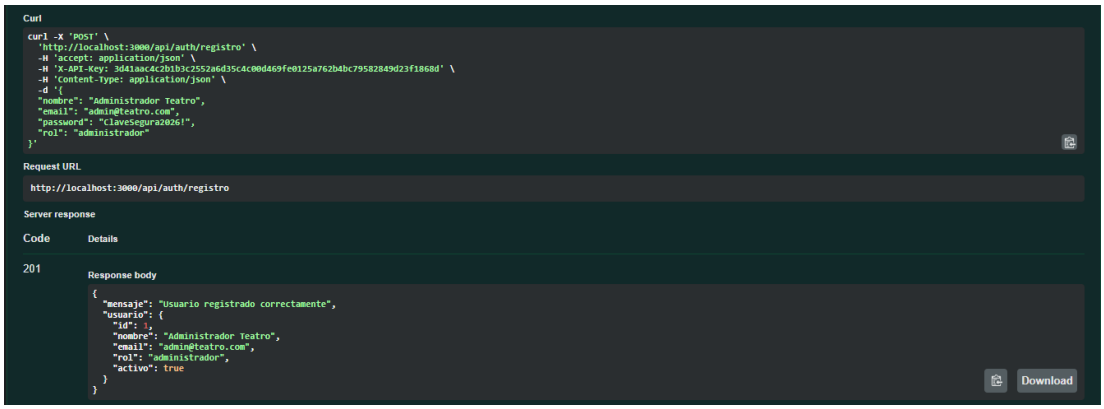


Figura 4. Registro del primer usuario. Responde 201 sin exponer password ni passwordHash.

Correo duplicado, 409

Al repetir exactamente la misma petición, el controlador detecta que el correo ya existe y responde 409.

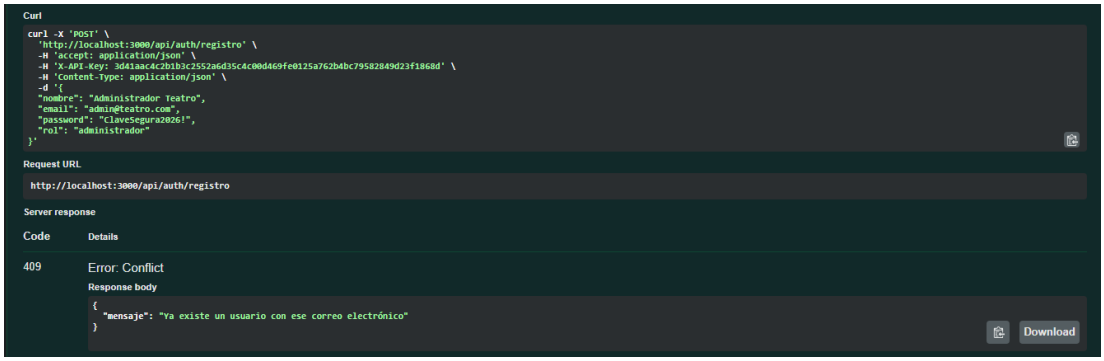


Figura 5. Intento de registro con un correo ya existente. Responde 409 Conflict.

Contraseña menor a 10 caracteres, 400

Si se intenta registrar un usuario con una contraseña corta, el validador corta la petición antes de llegar al controlador.

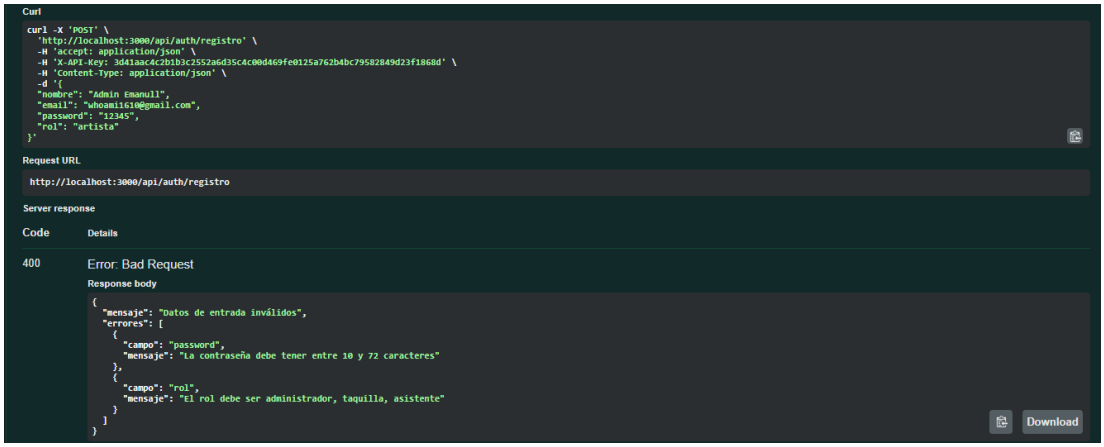


Figura 6. Rechazo por validación cuando la contraseña tiene menos de 10 caracteres.

Login fallido y enumeración de usuarios, 401

Probamos el login en dos escenarios de fallo: primero con el correo correcto y una contraseña equivocada, y después con un correo que no está registrado.

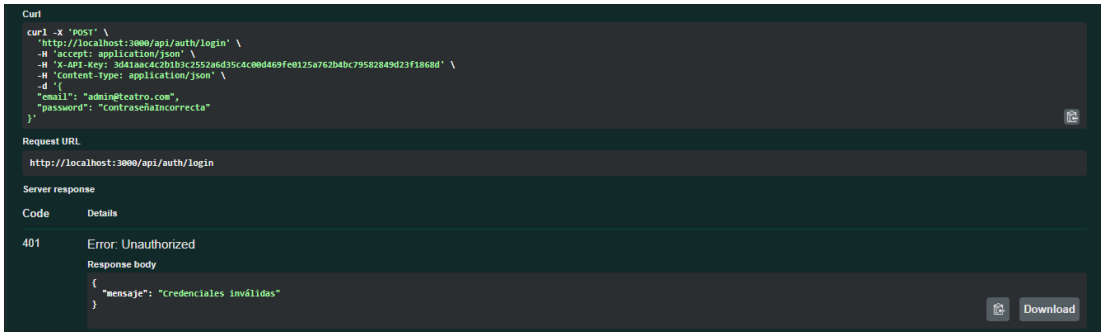


Figura 7. Login con contraseña incorrecta. Responde 401, "Credenciales inválidas".

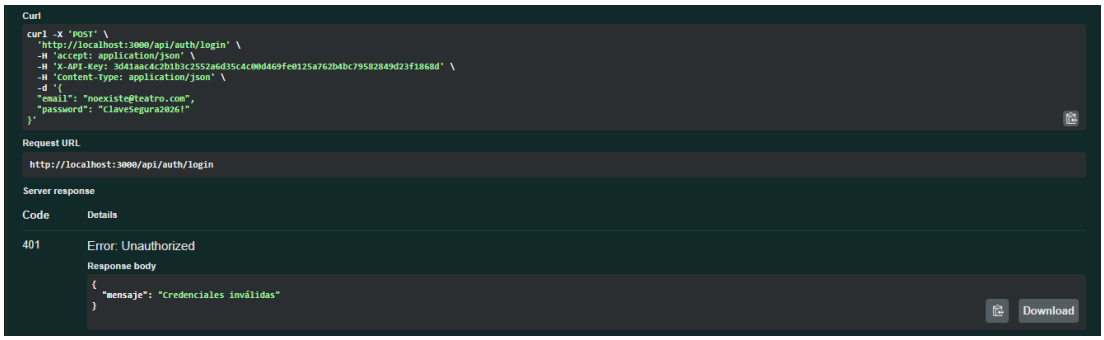


Figura 8. Login con un correo inexistente. Responde el mismo 401 y el mismo mensaje, sin revelar qué falló.

En los dos casos la respuesta es idéntica. Es a propósito: si el servidor dijera "el usuario no existe" en un caso y "la contraseña está incorrecta" en el otro, cualquiera podría probar una lista de correos y averiguar cuáles tienen cuenta en el teatro. Eso se llama enumeración de usuarios.

Login correcto, 200

Al enviar el correo con su contraseña real, `bcrypt.compare` confirma la coincidencia contra el hash guardado y el endpoint responde 200.

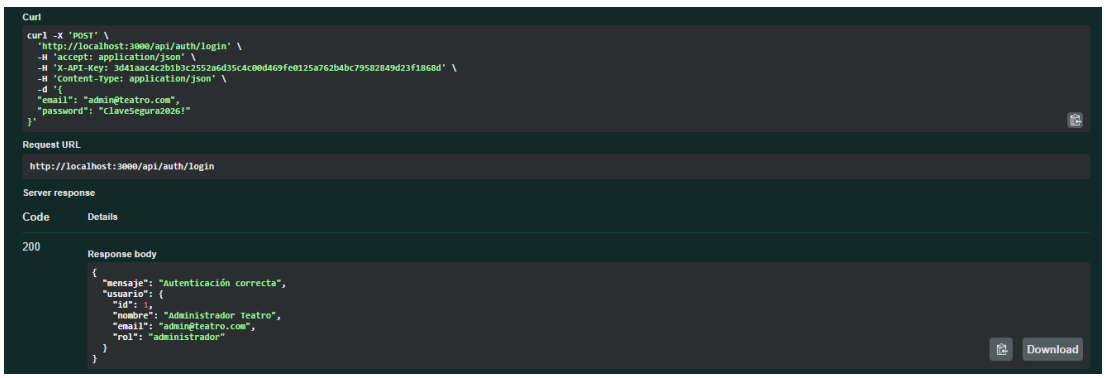


Figura 9. Inicio de sesión correcto. Responde 200 con los datos públicos del usuario.

6. La prueba de la sal

Para comprobar en la práctica que bcrypt usa una sal distinta en cada registro, creamos un segundo usuario con exactamente la misma contraseña que el primero, `ClaveSegura2026!`, pero con otro correo y otro rol.

El problema es que el hash no sale nunca en la respuesta, así que desde fuera no hay forma de verlo. Lo resolvimos como indica el laboratorio: pusimos un `console.log` temporal dentro de `crearUsuario`, registramos los dos usuarios y comparamos lo que salió por consola.



Figura 10. Dos usuarios con la misma contraseña generan hashes distintos, porque cada uno lleva su propia sal.

Los dos hashes comparten el prefijo `$2b$12$`, que indica la variante de bcrypt y el factor de costo. A partir de ahí son completamente distintos, empezando por la sal:

```
$2b$12$nnm2NM1nqz3pv3QFV5c3Oe...$2b$12$C/sxWpMjx3OV8msTMSbfhO...
```

Misma contraseña de entrada, hashes de salida distintos. Eso es exactamente lo que hace la sal, y es lo que impide que un atacante se dé cuenta de que dos personas usan la misma clave.

Terminada la prueba quitamos el `console.log`. Los registros normales del servidor no deben guardar hashes de credenciales, aunque sean hashes.

7. Mass assignment

Probamos enviar en el registro un cuerpo con campos de más, de los que un atacante usaría para alterar el estado interno de la cuenta:

```
"activo": false, "esSuperAdmin": true, "passwordHash": "HASH_FALSO"
```

Aquí actúan dos defensas encadenadas. Primero `matchedData` descarta todo campo que no esté declarado en el validador, así que esos tres ni siquiera llegan al service. Y aunque llegaran, el service calcula siempre el hash con bcrypt y fija `activo: true` por su cuenta.

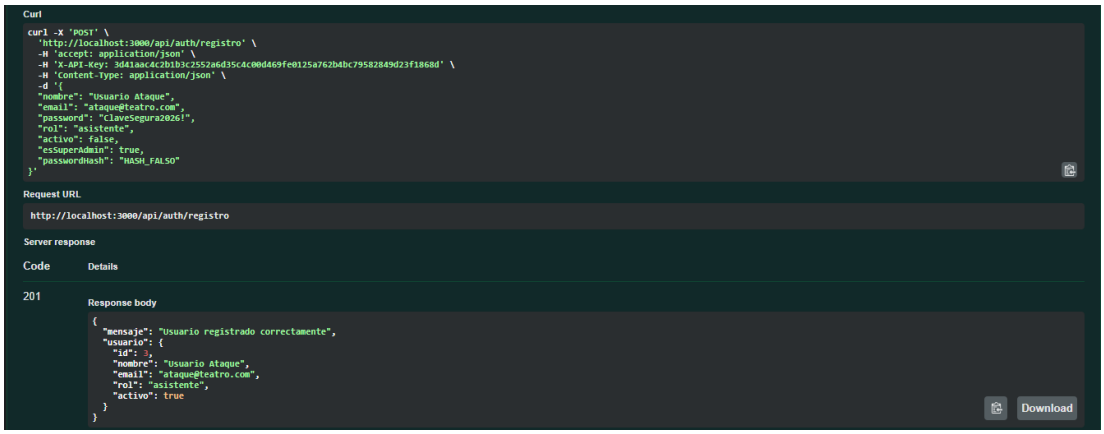


Figura 11. El usuario se crea con activo true y sin los campos inyectados. El id también lo asigna el servidor, no el cliente.

8. Los usuarios viven en memoria

Detuvimos el servidor con Ctrl + C y lo volvimos a levantar. Como `usuarios.js` mantiene las cuentas en un array en memoria, al reiniciar vuelve a quedar vacío, y un login con el usuario que acabábamos de crear respondió 401.

Es una limitación conocida, no un fallo. La estructura ya está lista para mover la persistencia a PostgreSQL en los siguientes laboratorios.

9. Lo que todavía no sabemos

Con este bloque ya tenemos los dos niveles de identificación: sabemos qué aplicación consume la API y podemos comprobar la identidad de una persona con correo y contraseña.

Pero quedan dos huecos.

En el registro, cualquiera con una API Key válida puede elegir el rol administrador o taquilla. Cerrar esa escalada de privilegios es lo que sigue.

Y el login solo responde que las credenciales son válidas, no emite ningún token. Por eso el resto de endpoints, los de eventos, funciones o boletas, todavía no saben qué usuario los está llamando ni restringen nada según el rol.

10. Repositorio

https://github.com/JuanK2550/API-TEATRO

La implementación está en el commit `098e76b`, y las salidas de Semgrep y npm audit del laboratorio en `d24cd27`. Las dos herramientas se repitieron al terminar: Semgrep corrió 73

reglas sobre 41 archivos con `0 hallazgos` y npm audit auditó los 146 paquetes con `found 0 vulnerabilities`.

Antes de subir comprobamos que el `.env` no estuviera en el repositorio, que ninguna API Key apareciera en ningún archivo y que no quedara ningún `console.log` con hashes. En esa revisión encontramos que el README había quedado con los dos hashes reales de la prueba de la sal, y los reemplazamos por un esquema.